WAS Experiment 07

Aim: Simulating CSRF Attacks

Learning Objective: To demonstrate a Cross-Site Request Forgery (CSRF) attack in a controlled web application environment and understand how an attacker can cause an authenticated user's browser to send an unintended request to a vulnerable application.

Learning Objectives:
After completing this experiment, students will be able to:

- Understand the concept of CSRF.
- Explain how CSRF attacks work.
- Understand the role of browser sessions and cookies in CSRF.
- Identify applications vulnerable to CSRF.
- Simulate a CSRF attack in a controlled laboratory environment.
- Understand the difference between CSRF and XSS.
- Understand the purpose of CSRF tokens.
- Apply security measures to prevent CSRF attacks.

Tools: Kali Linux, Terminal, Burp Suite, Web Browser-Firefox, HTML editor / Visual Studio Code and Local or authorized vulnerable web application.

Theory:

Introduction

Cross-Site Request Forgery (CSRF) is a web application vulnerability in which an attacker tricks an authenticated user's browser into sending an unintended request to a web application.
The attack takes advantage of the fact that browsers automatically include relevant authentication information, such as session cookies, with requests to a website.
For example, if a user is logged into a web application and visits a malicious page, that page may attempt to cause the user's browser to send a request to the application.

How CSRF Works

A CSRF attack generally requires:

1. The victim must be authenticated to the target application.
2. The application must rely on automatically submitted authentication credentials, such as cookies.
3. The application must lack sufficient protection against forged requests.
4. The attacker must be able to cause the victim's browser to submit a request.

For example, consider a vulnerable application that changes a user's email address using:
POST /change-email
with:
email=newemail@example.com
If the application does not verify that the request originated from a legitimate form, an attacker may attempt to cause an authenticated user's browser to submit the request.

CSRF Token

A CSRF token is a unique, unpredictable value associated with a user's session or request.

For example:
<input type="hidden" name="csrf_token" value="random-token-value">
The server verifies the token before processing the state-changing request.
An attacker generally cannot successfully forge the request without knowing the valid token.

Prevention Techniques
CSRF can be mitigated using several security mechanisms:

- Use anti-CSRF tokens for state-changing requests.
- Use appropriate SameSite cookie attributes.
- Validate the Origin and/or Referer headers where appropriate.
- Require re-authentication for highly sensitive operations.
- Avoid using GET requests for state-changing operations.
- Implement proper server-side authorization checks.
- Use secure framework-provided CSRF protection mechanisms.

Procedure:

Step 1: Start Kali Linux

Start Kali Linux and open Burp Suite.

burpsuite

Select Temporary Project and continue with the default configuration.

Expected Output:

Burp Suite starts successfully.

Step 2: Configure Firefox

Configure Firefox to use Burp Suite as a proxy:

Proxy Address: 127.0.0.1

Proxy Port: 8080

Expected Output:

Firefox traffic is routed through Burp Suite.

Step 3: Open the Authorized Web Application

Open a local or intentionally vulnerable web application designed for CSRF testing.

Example:

http://localhost/

Only use an application that has been specifically authorized for security testing.

Expected Output:

The web application's login page is displayed.

Step 4: Log In as a Test User

Use the credentials provided by the laboratory application.

For example:

Username: testuser

Password: testpassword

Log in successfully.

Expected Output:

The user is authenticated and the application creates a session.

Step 5: Identify a State-Changing Function

Identify a function that changes application data.

Examples include:

- Change email address
- Change password
- Update profile information
- Add an address
- Change account settings

For this experiment, use a harmless test operation such as changing a test account's profile information.

Expected Output:

A state-changing form or request is identified.

Step 6: Capture the Request Using Burp Suite

Open:

Proxy → Intercept

Perform the selected action in the application.

Burp Suite captures the request.

For example:

POST /change-email HTTP/1.1

Host: localhost

Cookie: session=ABC123

email=test@example.com

Expected Output:

The authenticated request is displayed in Burp Suite.

Step 7: Analyze the Request

Examine the captured request and identify:

- HTTP method
- URL
- Session cookie
- Parameters
- CSRF token, if present

Check whether the request contains a CSRF protection mechanism.

Expected Output:

The authentication information and request parameters can be identified.

Simulating the CSRF Attack

Step 8: Create a Test CSRF Page

In the controlled laboratory environment, create a simple HTML page that attempts to submit a request to the vulnerable application.

Example structure:

```
<form action="http://localhost/change-email" method="POST">
  <input type="hidden" name="email" value="csrf-test@example.com">
  <input type="submit" value="Submit">
```

</form>

This should only be used against the intentionally vulnerable laboratory application.

Expected Output:

The test page contains a form that targets the application's state-changing endpoint.

Step 9: Access the Test Page While Authenticated

Keep the test user logged into the vulnerable application.

Open the CSRF test page in the same browser session.

Submit the test form.

Expected Output:

If the application is vulnerable, the browser sends the request while the user's authenticated session is still active, and the application processes the action.

Step 10: Verify the Result

Return to the target application's profile or account page.

Check whether the test change was successfully applied.

Expected Output:

The state-changing operation is performed without the user intentionally submitting the legitimate application form.

This demonstrates the basic concept of CSRF.

Testing CSRF Protection

Step 11: Test a Protected Application

Repeat the experiment against a version of the application that implements CSRF protection.

Capture the legitimate request using Burp Suite.

Look for a CSRF token, such as:

csrf_token=random-value

Expected Output:

The legitimate request contains a CSRF token.

Step 12: Submit the Forged Request Without a Valid Token

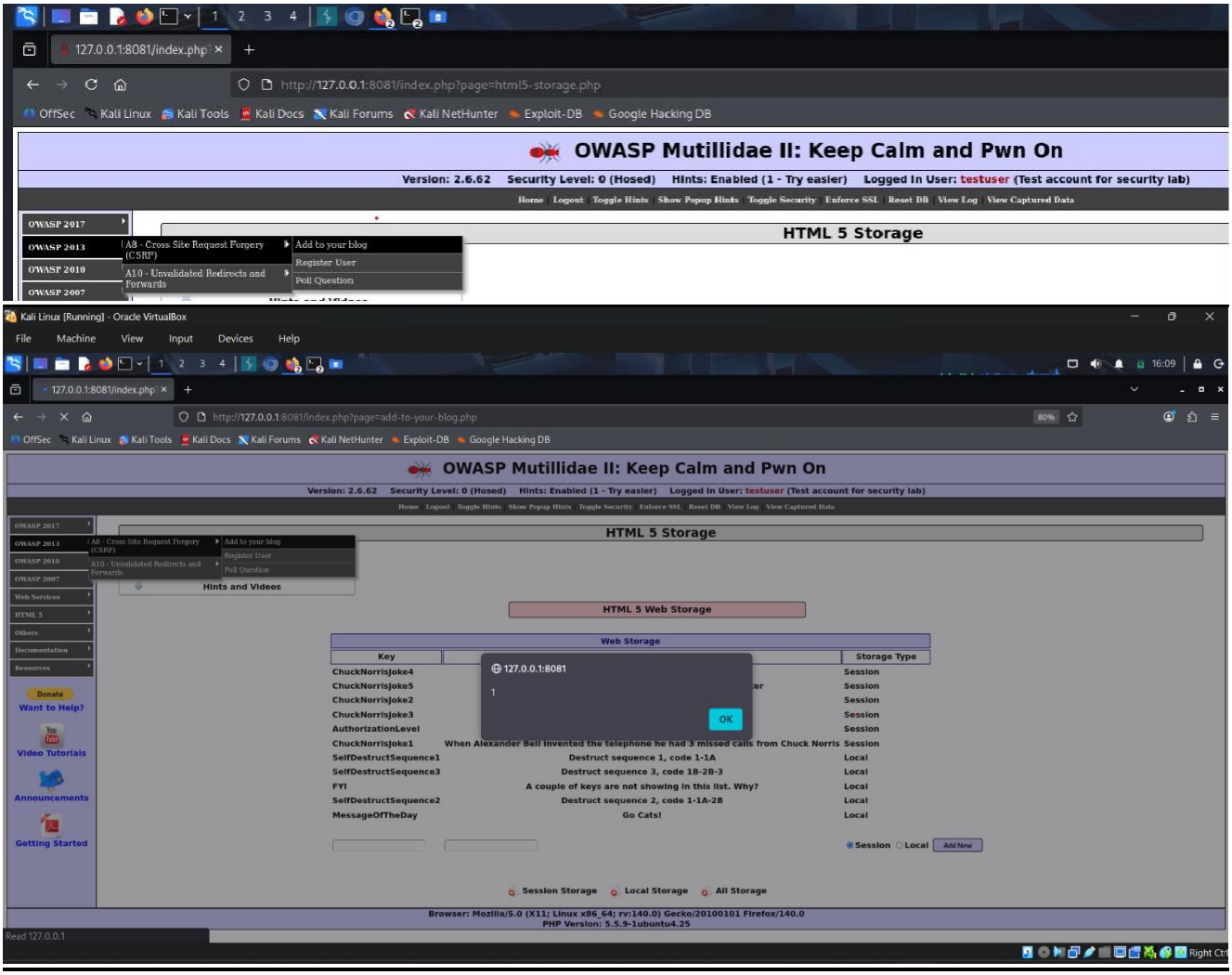
Attempt the same controlled test without supplying the valid CSRF token.

Expected Output:

The server rejects the forged request.

This demonstrates how CSRF tokens help prevent unauthorized cross-site requests.

Outputs:



Observation:

The following observations are made:

1. The victim must generally have an authenticated session for the basic CSRF scenario.
2. Browsers automatically include applicable authentication information with requests.

3. A vulnerable application may process a forged state-changing request.
4. Burp Suite can be used to capture and analyze the legitimate request.
5. CSRF tokens can be used to distinguish legitimate requests from forged requests.
6. A protected application should reject requests containing an invalid or missing CSRF token.
7. CSRF primarily targets state-changing actions performed using a victim's authenticated session.

Result and Discussion:

The experiment successfully demonstrated the concept of a Cross-Site Request Forgery attack in a controlled laboratory environment.

A state-changing request was captured using Burp Suite and analyzed to determine the authentication mechanism and CSRF protection. A controlled CSRF test was then performed against the intentionally vulnerable application.

The experiment demonstrated that an application that relies solely on browser-supplied session credentials and does not adequately verify the legitimacy of state-changing requests may be vulnerable to CSRF.

The protected version of the application rejected the forged request when the required CSRF protection was absent or invalid.

Conclusion:

This experiment demonstrated the concept of **Cross-Site Request Forgery (CSRF)** by simulating a forged request against an intentionally vulnerable web application.

The experiment showed how an authenticated user's browser can be induced to send an unintended state-changing request and demonstrated how CSRF tokens and appropriate cookie protections can prevent such attacks.

CSRF testing should always be conducted in a controlled environment or against applications for which explicit authorization has been obtained.

Viva Questions:

1. What is Cross-Site Request Forgery (CSRF)?
2. How does a CSRF attack work?
3. What is the purpose of a CSRF token?

For Faculty Use:

Correction Parameters	**Formative Assessment [40%]**	**Timely completion of Practical [40%]**	**Attendance / Learning Attitude [20%]**	
Marks Obtained				